Deepfake ML Hackathon: Technical Report

This report summarizes the submitted model, the checks run on the organizer data, and the limitations of those checks. Results below come from the local scripts and dataset described here.

1. Problem, dataset, and compliance

Objective

The task is to classify each face image as real (`label = 0`) or fake (`label = 1`). The submitted PyTorch model uses EfficientNet-B0 and produces a confidence value interpreted as the probability that an image is fake. The hackathon's primary metric is ROC AUC, which measures how well those confidence values rank fake images above real ones across thresholds.

Organizer-provided data

The organizer archive contains 7,000 labeled training images and 3,000 unlabeled test images. The training set has 3,478 real images and 3,522 fake images. All 7,000 IDs in the training metadata matched image filenames. The 3,000 test image IDs also matched the IDs in the supplied sample submission.

The data layout is:

```
hackathon_dataset/
 train/
 real/
 fake/
 test/
 images/
 train_metadata.csv
 sample_submission.csv
```

Validation used only the provided training images; no external training data was added. The model starts from the permitted ImageNet-pretrained EfficientNet-B0 and is fine-tuned on the organizer's training set. Test images were used only to generate predictions and check the submission format. They were not used for training or validation.

Before evaluation, the archive contents were checked against the metadata. Every metadata ID had a matching image, and every training image was listed in the metadata. Files in `train/real/` corresponded to label 0; files in `train/fake/` corresponded to label 1. The sample submission also contained the same 3,000 IDs as the test image folder.

Validation protocol

The validation subset is drawn from the training metadata using the organizer-recommended split:

```
train_test_split(
 metadata,
 test_size=0.2,
 stratify=metadata["label"],
 random_state=42,
)
```

The split contains 1,400 images: 696 real and 704 fake. Stratification keeps the class balance similar to the full training set. Test labels are private, so test accuracy and test ROC AUC cannot be calculated from the provided files.

The order of rows passed to `train_test_split` matters. The evaluation script uses the order in `train_metadata.csv`; a different row order can select a different validation subset, even with the same split fraction, stratification, and seed. Future comparisons should keep the metadata order fixed or save the validation IDs with the results.

Label and confidence conventions

The label convention is the same throughout the metadata and submission: real is 0 and fake is 1. The `confidence` column is the model's sigmoid probability for the fake class. The organizers use this value with hidden labels to calculate AUC. The binary label is 1 when confidence is at least 0.5. Changing the threshold would affect accuracy and F1, but not ROC AUC computed from the confidence values.

2. Model and training pipeline

Architecture

The network uses torchvision's EfficientNet-B0 with its ImageNet-pretrained feature extractor. The original classification head is replaced with:

Component	Configuration
Feature backbone	EfficientNet-B0
Regularization	Dropout, probability 0.3
Output layer	Linear, 1,280 input features to 1 logit
Output interpretation	Sigmoid probability for fake class
Total parameters	4,008,829

EfficientNet-B0 is the compact backbone used by the submitted checkpoint and inference script. No architecture comparison was run for this submission. ImageNet initialization is allowed by the event rules; fine-tuning and validation use only the provided dataset.

The final layer emits one logit. Training uses `BCEWithLogitsLoss`, which combines the sigmoid operation and binary cross-entropy in a numerically stable loss. At inference, sigmoid converts the logit to a fake-class confidence in `[0, 1]`; the script assigns label 1 at confidence `>= 0.5`.

Image processing and augmentation

Images are converted to RGB, resized to 224 by 224 pixels, converted to tensors, and normalized using the standard ImageNet channel means and standard deviations. Training augmentation applies random horizontal flips and brightness/contrast jitter with magnitude 0.1. Gaussian blur is applied to 20% of training examples, with a radius sampled from 0.5 to 1.5. Validation and test preprocessing do not apply random augmentation.

Optimization

Setting	Value
Optimizer	AdamW
Learning rate	0.0003
Weight decay	0.0001
Batch size	32
Epochs	10
Random seed	42
Checkpoint selection	Highest validation ROC AUC

The settings above are those recorded in the training script. Dropout and image augmentation are used to regularize fine-tuning. No controlled hyperparameter comparison is included, so these values have not been shown to be optimal.

The training log marks epoch 8 as the best checkpoint and records a clean validation AUC of 0.8925. Training loss falls from 0.5270 at epoch 1 to 0.0811 at epoch 10, while the logged validation AUC remains around 0.88-0.89. The gap between falling training loss and mostly flat validation AUC suggests possible overfitting.

The training script seeds Python, NumPy, and PyTorch with 42 and uses a stratified split. Random augmentation is limited to training; validation preprocessing is deterministic. Training uses automatic mixed precision when CUDA is available and regular precision otherwise. The saved checkpoint is a PyTorch state dictionary loaded into the same EfficientNet-B0 structure for inference. There is no learning-rate scheduler in the recorded configuration. The artifacts include neither repeated-seed runs nor an ablation of augmentations, regularization, or optimizers. The settings describe the pipeline that was run; they are not evidence that each choice is optimal. Any future comparison should use fixed validation IDs and leave the test set untouched.

3. Measured validation and robustness diagnostics

The existing checkpoint was evaluated with `evaluate_validation.py` on the 1,400-example split from `train_metadata.csv`. The checkpoint was not retrained or replaced for this run. ROC AUC, accuracy, and F1 were calculated from the model probabilities and predictions at threshold 0.5.

Validation input	ROC AUC	Accuracy	F1
Clean	0.9861	0.9636	0.9634
JPEG quality 50	0.9822	0.9571	0.9573
Gaussian blur, radius 1	0.9840	0.9557	0.9549
Downsample by half, then resize	0.9803	0.9457	0.9442

The corruption rows are local checks on the validation images. Results stayed similar under these specific transformations, but they are not the hidden test variants and are not the official robustness score. The checkpoint was selected using validation AUC, so these results are also subject to model-selection bias.

ROC AUC measures how often fake images receive higher fake-class confidence than real images across thresholds, matching the organizers' primary metric. Accuracy is the share of thresholded labels that are correct. F1 is the harmonic mean of precision and recall for the fake class. The training classes are nearly balanced, so accuracy and F1 add context, but ROC AUC remains the main comparison. These are validation metrics; they do not reveal the hidden test score.

The transformations are deterministic. JPEG quality 50 re-encodes each RGB image in memory. Gaussian blur uses radius 1.0. The half-resolution check reduces each image dimension by half with bilinear interpolation, then restores the original size before the standard 224-by-224 transform. Every variant uses the same validation IDs as the clean run. These checks are repeatable, but cover only a few of the image changes that may occur in practice.

Relative to the clean validation row, the AUC decreased by about 0.0039 for JPEG quality 50, 0.0021 for blur, and 0.0058 for half-resolution resizing. Accuracy decreased by about 0.0064, 0.0079, and 0.0179 respectively. These are absolute differences calculated from the displayed measurements, not official robustness scores. The largest of these particular changes affected thresholded accuracy on half-resolution images; this observation should not be generalized to undisclosed transformations or other datasets.

Historical metric discrepancy

The existing `training_log.txt` records validation AUC 0.8925. Evaluating the supplied checkpoint on the metadata-ordered split produced 0.9861. Building the split from class-directory iteration order selected different images and produced 0.9823. A fixed seed does not guarantee the same split if the input order changes. The available files do not explain why the historical training score differs from both later measurements. The report keeps all three values with their context rather than treating them as interchangeable.

Efficiency observations

The model has 4,008,829 parameters. On this CPU, clean validation took 54.33 seconds for 1,400 images, or about 25.77 images per second including loading and preprocessing. This is not the organizers' NVIDIA T4 benchmark. Inference uses batches of 64 and enables CUDA autocast when a CUDA device is available. The 3,000-image run completed locally, but its runtime was not measured on a T4.

The validation run used Python 3.10.11, PyTorch 2.14.1+cpu, torchvision 0.29.2+cpu, pandas 2.3.3, scikit-learn 1.7.2, and Pillow 12.2.0. CUDA was unavailable. These versions differ from the environment in the original project notes. `requirements.txt` contains the project's pinned installation versions. Runtime and small numerical differences may vary by hardware and library version.

4. Test predictions, limitations, and reproduction

Test-set prediction verification

The inference script processed all 3,000 test images and wrote a new `submission_predictions.csv`, leaving the existing `predictions.csv` unchanged. The checker verified the required ordered columns `image_id,label,confidence`, all 3,000 image IDs, binary labels, and finite confidence values between 0 and 1. The existing `predictions.csv` also passes the checker against the organizer test directory. Its IDs and labels match the newly generated output; the largest absolute confidence difference was approximately 0.000003, consistent with minor numerical differences across inference runs. Since test labels are private, these checks verify format and coverage only, not prediction correctness.

The checker confirms that every test filename has one row, that the header and column order are correct, and that labels and confidence values are well-formed. It cannot tell whether a prediction matches a private test label. Agreement with the existing CSV helps verify reproducibility, but is not an independent performance measurement.

Limitations

? Test labels and hidden test corruptions are unavailable, so test accuracy

and the official robustness score cannot be reported.

? Local corruption diagnostics cover only JPEG quality 50, Gaussian blur at

radius 1, and half-resolution resizing; other distortions may behave differently.

? The training log's AUC does not match either later evaluation of the

supplied checkpoint. The available files do not explain why.

? No controlled ablation was recorded for architecture, augmentations,

threshold, or optimizer settings.

? The speed measurement was CPU-only and does not predict the result on the

organizers' T4 hardware.

Reproduction

From the repository directory, after extracting the provided archive beside the scripts:

```
python -m pip install -r requirements.txt
python .\evaluate_validation.py --data-dir .\hackathon_dataset --weights .\model_weights.pt
python .\inference_script.py --test_dir .\hackathon_dataset\test\images --weights .\model_weights.pt
--output .\submission_predictions.csv
python .\check_submission.py --test_dir .\hackathon_dataset\test\images --predictions .
\submission_predictions.csv
```

The evaluator prints split metrics. The checker validates the output without changing the predictions. For the Drive submission, include the PyTorch weights, verified predictions, source, requirements, and this report. Set Drive access to **Anyone with the link — Viewer** and check it in a private browser window before submitting.

To reproduce training in the recorded Linux/Colab-style setup, extract the archive and run the existing training script with the dataset root as the working directory:

```
cd hackathon_dataset
python ../train.py
```

The script reads `./train/real/` and `./train/fake/` relative to the current directory and writes `model_weights.pt` there. Running it from `hackathon_dataset/` keeps a new checkpoint separate from the submitted one beside the project scripts. The evaluation and inference commands above load the submitted checkpoint and write predictions to a separate file.

AI assistance

AI assistance was used in preparing this project and its documentation. The team should follow the hackathon's rules on AI use and disclose assistance as required.